Agent RAG agentique avec LangGraph — cybersécurité et réglementation numérique

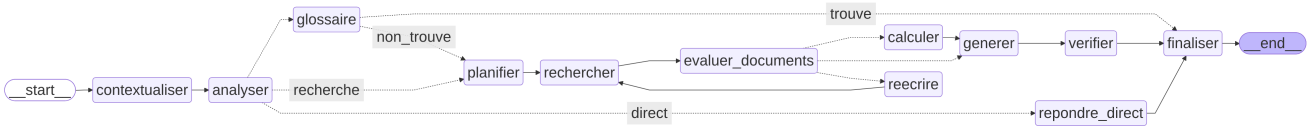
Module SMA et IAD — Pr. Sara RETAL — ENSET Mohammedia — dépôt GitHub : agentic-rag-langgraph
Étudiant : Mohammed ELJIHAD — Master II-BDCC

L'objectif est un agent capable de répondre à des questions simples et complexes (comparaisons, questions multi-documents, calculs) à partir d'une base documentaire choisie. Le domaine retenu est la **cybersécurité et la réglementation numérique** (RGPD, NIS2, lois marocaines 09-08 et 05-20, ISO 27001, menaces et bonnes pratiques). Le système est un **graphe LangGraph explicite** (StateGraph, arêtes conditionnelles, boucle, checkpointer) sans agent préconstruit, exécuté entièrement en local sur CPU (2 cœurs, 7 Go de RAM, sans GPU ni clé API).

1. Démarche : données, modèles et outils

- **Base documentaire** : 15 documents Markdown en français, **rédigés par l'auteur** (synthèses pédagogiques de textes publics, sans reproduction de contenus protégés), avec métadonnées (id, titre, catégorie), et un glossaire de 34 acronymes.
- **Prétraitement** : suppression de la syntaxe Markdown, intertitres conservés comme préfixes (« Notification : … »), découpage par paragraphes puis par phrases (cible 650 caractères, recouvrement 120), ce qui donne **58 fragments** sans phrase coupée.
- **Vectorisation** : `intfloat/multilingual-e5-small` (384 dimensions, préfixes query/passage, vecteurs normalisés) et index **FAISS** IndexFlatIP (cosinus).
- **LLM** : interface unique `chat()` avec deux implémentations : un modèle local *transformers* et toute API compatible OpenAI (Groq, OpenAI, Ollama…) via variables d'environnement. Benchmarks réalisés : Qwen2.5-1.5B en float32 → mémoire saturée ; en bfloat16 → 2,4 tokens/s ; int8 dynamique complet → sorties incohérentes (valeurs aberrantes d'activation). Solution retenue : **Qwen2.5-1.5B-Instruct avec quantification int8 des seules projections gate/up des MLP**, soit 3,4 tokens/s avec des sorties correctes. Qwen2.5-0.5B (6 tokens/s) a été écarté : ses réponses mélangeaient les sources.
- **Outils** (décorateur LangChain `@tool`) : `rechercher_documents` (FAISS, filtre de catégorie), `calculatrice` (évaluation sûre par AST, sans eval), `consulter_glossaire`, `lister_documents`.

2. Fonctionnement du graphe



L'**état** (TypedDict) contient les `messages` (réducteur add_messages, persistés par le checkpointer **MemorySaver** selon le `thread_id`), la question autonome, la route, les sous-questions, les fragments avec leur verdict de pertinence, le compteur de réécritures, le calcul, la réponse, les sources et une trace horodatée de chaque nœud. Toutes les décisions du LLM sont analysées de façon déterministe et disposent d'un repli, ce qui rend le système robuste avec un petit modèle.

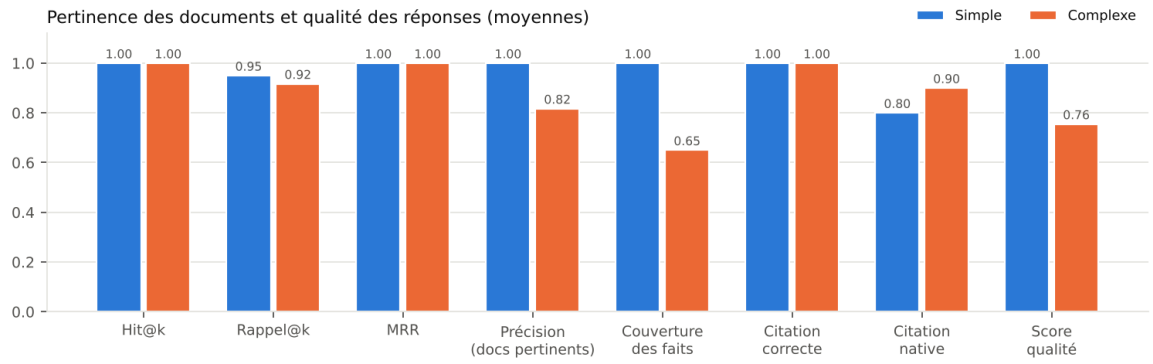
Nœud	Rôle agentique
contextualiser	Mémoire : réécrit une question de suivi (« Et quelle amende… ? ») en question autonome à partir de l'historique du fil.
analyser	Routeur hybride : règles (salutation, glossaire, complexité, quantités à calculer) + classification LLM + garde-fou par similarité (si le LLM répond « direct » mais que la base contient un passage proche, on force la recherche).

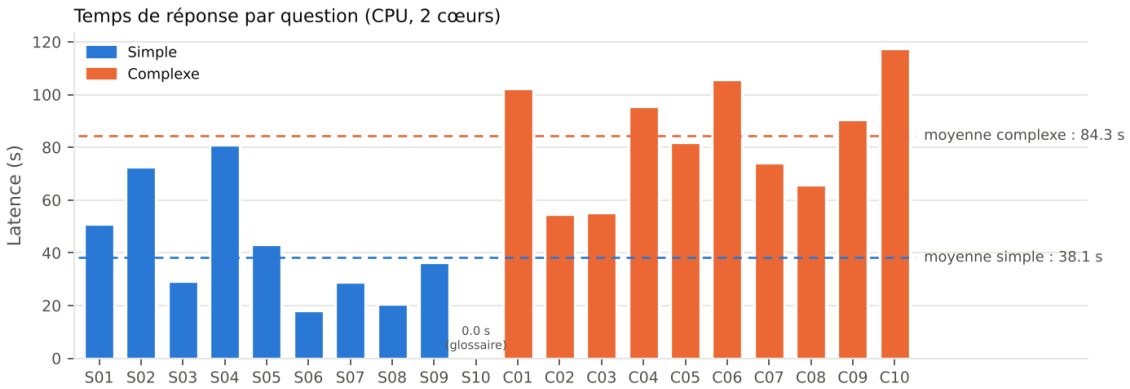
Nœud	Rôle agentique
planifier	Décompose une question complexe en 2–3 sous-questions (few-shot, analyse ligne à ligne, repli par découpage) ; la question entière est toujours ajoutée aux requêtes.
rechercher / evaluer_documents	Recherche par sous-requête (top-4), fusion et dédoublonnage ; pertinence par score (≥ 0,88 gardé, < 0,82 rejeté) et jugement OUI/NON du LLM en zone grise ; le meilleur fragment de chaque requête est conservé.
reecrire (boucle)	Si une sous-question n'a aucun fragment pertinent : reformulation (LLM + expansion des acronymes), puis retour vers rechercher (2 boucles max, arrêt si aucun fragment nouveau).
calculer	Le LLM traduit le problème en expression (few-shot) ; l'expression doit contenir une opération et un nombre de la question ; sinon une seconde tentative guidée est faite.
generer / verifier / finaliser	Réponse citée [Dxx] à partir d'un contexte diversifié par sous-question et élargi aux fragments voisins ; vérification déterministe (citations valides, doublons, résultat du calcul présent, indicateur d'ancrage) ; écriture dans la mémoire.

3. Évaluation

Protocole : 10 questions simples et 10 complexes (comparaisons, multi-documents, 3 calculs), chacune avec ses sources attendues et ses faits clés (`eval/questions.json`). Chaque question est exécutée réellement dans un fil séparé, et toutes les mesures sont produites par `src/evaluate.py`. **Pertinence des documents** : Hit@k, Rappel@k, MRR, et précision des documents jugés pertinents. **Qualité** : couverture des faits clés dans la réponse (comparaison normalisée) et citation d'une source attendue ; **score qualité = 0,7 × couverture + 0,3 × citation**, complété par un **jugement manuel** (correct / partiel / incorrect). **Temps** : latence de bout en bout, chargement exclu (34,8 s).

Groupe	Latence moy. (s)	Hit@k	Rappel@k	MRR	Précision pert.	Couverture faits	Citation (native)	Score qualité	Jugement manuel
Simples (10)	38,1	1,00	0,95	1,00	1,00	1,00	1,00 (0,80)	**1,00**	10 corrects
Complexes (10)	84,3	1,00	0,92	1,00	0,82	0,65	1,00 (0,90)	**0,76**	4 C / 5 P / 1 I
Global (20)	61,2	1,00	0,93	1,00	0,90	0,83	1,00 (0,85)	**0,88**	14 / 5 / 1





Analyse. La recherche est très fiable sur ce corpus (Hit@k = 1 et MRR = 1 : le premier document récupéré est toujours une source attendue). Les questions simples sont toutes correctes. Les deux calculs réglementaires sont exacts grâce à l'outil (C02 : max(20 M€ ; 4 % × 1 Md€) = 40 M€ ; C03 : 28 M€). Les faiblesses portent sur la **synthèse multi-documents** : les documents nécessaires sont retrouvés et jugés pertinents, mais le modèle de 1,5 B ne traite qu'une partie de la question (C01 : délais NIS2 omis ; C05 : loi 05-20 omise) ou attribue un passage à la mauvaise source (C08, C09). C10, qui demandait un raisonnement chiffré implicite, est faux (la calculatrice a rejeté l'expression « 24 ») ; la métrique automatique le crédite pourtant de 0,65, ce qui montre ses limites. La latence (≈ 3,4 tokens/s) est dominée par la génération : 6,9 appels LLM par question complexe.

Itérations guidées par l'évaluation (résultats conservés) : en v1, le contexte coupait des listes (S04) et la calculatrice acceptait une valeur seule. Le contexte a donc été diversifié et élargi aux fragments voisins, et l'expression est désormais validée : score qualité 0,837 → 0,878. En v2, les requêtes reformulées n'étaient pas rattachées à leur sous-question, ce qui provoquait des boucles inutiles. Avec le suivi des origines et l'arrêt sur stagnation, les réponses sont identiques mais la latence des questions complexes passe de 94,7 s à 84,3 s. La boucle de réécriture fonctionne hors jeu de test : « OIV obligations » donne 0/4 fragment pertinent, puis reformulation, nouvelle recherche et 1 fragment pertinent. La mémoire a été vérifiée sur deux tours : « Et quelle amende maximale… ? » est réécrite en « Quelle est l'amende maximale encourue pour un manquement grave selon le RGPD ? », et la réponse cite D06.

4. Limites et améliorations

- **Modèle** : un LLM de 1,5 B sur CPU limite la synthèse multi-sources et impose ≈ 1 à 2 minutes par question complexe. Le code accepte déjà un modèle plus puissant via une API compatible OpenAI ; il serait aussi possible d'utiliser llama.cpp/GGUF ou un GPU.
- **Récupération** : corpus petit et homogène, donc les scores e5 sont resserrés (0,78–0,93) et les seuils sont fixés empiriquement. Améliorations possibles : recherche hybride BM25 + vecteurs, reranker cross-encoder, corpus plus large et réel.
- **Agentivité** : le routeur LLM s'est trompé 2 fois sur 19 (corrigé par le garde-fou) et les reformulations peuvent être inexactes (« OIV » développé en « OIF »). Pistes : appel de fonctions natif (bind_tools) avec un modèle plus grand, et un nœud de critique qui vérifie que chaque sous-question est traitée dans la réponse.
- **Évaluation** : 20 questions sur un corpus rédigé par l'auteur ; métrique par mots-clés approximative (faux positif C10). Il faudrait un jeu plus large, une évaluation LLM-juge ou RAGAS (fidélité, pertinence de la réponse) et plusieurs annotateurs.